

Certified lower bounds for van der Waerden numbers from a GPU scan and recurrence closure

Brice Pouly

Qualification status (28 August 2026). This is a private prepublication draft, not the citable release. The final 13 claim manifests, 515-chunk campaign recovery, ordered-prime audit, CUDA qualification, four external replications, outbound licences, and DOI/tag freeze remain open as recorded in `STATUS.json`. Numerical statements below report the current computation; release-level reproducibility claims remain conditional on those gates.

Abstract

We report outputs from a historical single-GPU campaign configured to scan power-residue colorings over all primes in $[9.7 \times 10^8, 2 \times 10^9]$. Its retained summaries identify candidate direct Rabung certificates for $W(3, k)$, $17 \leq k \leq 25$, headed by $W(3, 17) > 31\,385\,622\,833$. After automatically closing the registered published and archived baselines under the recurrences of Blankenship–Cummings–Taranchuk and Xu, these candidate direct values rank above every bound located in that corpus for $17 \leq k \leq 21$; the rows $22 \leq k \leq 25$ are direct-method improvements, not general van der Waerden records. We also report historical independent acceptances of four archived two-colour certificates; release-level rechecks remain an explicit gate. Applying the Blankenship–Cummings–Taranchuk recurrence to them gives $W(3, 25) > 628\,673\,328\,287$ and, respectively for $k = 26, 27, 28$, the further bounds $1\,221\,912\,599\,848$, $2\,068\,976\,009\,617$, and $2\,159\,051\,058\,266$. Each direct computational claim is represented by a succinct Rabung certificate (p, r, k) whose hypotheses can be re-established by a stand-alone verifier. As a secondary empirical result, one frozen forecast is compared with a cumulative holdout trajectory at four predetermined checkpoints; the disjoint increments are the primary summary and all Poisson calculations are reported conditionally on that model.

Provenance. The scanning software, historical campaign, analysis, and first draft were produced in collaboration with Anthropic’s Claude (models `claude-fable-5` and `claude-opus-4-8`). OpenAI Codex in ChatGPT Work subsequently contributed to publication-package engineering, contract hardening, automated audits, CPU reruns, and manuscript revision. All systems were operated under the author’s direction. Every direct computational claim reported here can be rechecked by a separate-source verifier that shares no code with the search engine and runs in time $O(p \log p)$. See the AI contribution statement (p. 20).

1 Introduction

For integers $r \geq 2$ (colours) and $k \geq 3$, the van der Waerden number $W(r, k)$ is the least N such that every r -colouring of $\{1, \dots, N\}$ contains a monochromatic arithmetic progression of length k . We put the number of colours first. Monroe [7] uses the reverse convention $W(k, r)$, while Fox and Hunter [2] write $w(k; r)$.

Two different mechanisms for lower bounds must be kept separate. Rabung’s power-residue construction [11] turns a suitable prime p into a direct bound

$$W(r, k) > (k - 1)p + 1.$$

Blankenship, Cummings and Taranchuk [1, Theorem 2.1], by contrast, prove the finite constructive recurrence

$$W(r, k) > q(W(r - \lceil r/q \rceil, k) - 1), \quad (1)$$

where q is the largest prime not exceeding k . It is therefore numerically relevant at the values of k considered here, not merely asymptotically. In particular, a documented strict bound $W(2, k) > B$ implies $W(3, k) > qB$. Xu’s concatenation recurrence [13] supplies another closure operation. We consequently assess priority only after closing a machine-readable registry of published, archived, and current bounds under both recurrences. The recent superexponential three-colour result of Fox and Hunter [2] is a separate asymptotic development.

Computational lower bounds based on power-residue colourings and their cyclic-zipper extensions have been pursued by Herwig, Heule, van Lambalgen and van Maaren [3], Rabung and Lotts [10], Liang, Xu, Shao and Xiu [4], and Monroe [7]. Monroe’s article describes the systematic scan limit as 9.5×10^8 , while its accompanying table lists a larger valid prime, 969 347 371. We use the tabled value as the numerical hand-off. The historical campaign was configured to scan every prime in $[9.7 \times 10^8, 2 \times 10^9)$ and to compute the active target profiles with $r \leq 9$; recovery of its complete checkpoint and ordered-prime evidence remains a release gate.

Our first contribution is a set of nine candidate direct three-colour Rabung certificates, for $17 \leq k \leq 25$. The headline candidate is

$$W(3, 17) > 31\,385\,622\,833$$

from $p = 1\,961\,601\,427$, a factor 2.02 over the previously published bound. Conditional on final acceptance of the claim manifests, recurrence closure leaves the direct bounds as the strongest located in the registered source corpus for $17 \leq k \leq 21$. For $22 \leq k \leq 25$ they remain direct Rabung-family champions but are dominated as general bounds. Section 4 reports both columns and their provenance rather than conflating the two notions of record.

Our second contribution is a candidate independent re-verification of four two-colour primes recorded in an archived phase of Monroe’s project [6], using a stand-alone checker and a fail-closed audit format. Historical runs accepted the four triples; the final raw manifests remain a release gate. Priority for the primes remains Monroe’s. Combining these candidate inputs with (1) gives the substantially stronger three-colour consequences for $25 \leq k \leq 28$ in Section 4. The archive states that the historical phase tested primes to four billion [8]; our checker tests the validity of its listed primes, not the exhaustiveness of that historical scan.

Our third contribution is methodological. Each direct result is a succinct certificate (p, r, k) , and a separate-source verifier re-establishes the hypotheses of Rabung’s theorem. Search paths that share arithmetic are described as redundant rather than independent, raw verdicts are parsed fail-closed, and the final claims table will be generated only from accepted manifests. We also retain a short empirical analysis of run-valid-prime counts. It concerns one correlated cumulative holdout trajectory, not four replications; disjoint increments are primary, and the Poisson calculations are explicitly conditional on the stated model and accompanied by retrospective dispersion diagnostics.

2 The Rabung criterion and certificates

2.1 Power-residue colorings

Throughout, $W(r, k)$ denotes the least N such that every r -coloring of $\{1, \dots, N\}$ contains a monochromatic arithmetic progression of length k . An *explicit certificate* for the lower bound $W(r, k) > n$ is an r -colouring of $\{1, \dots, n\}$ with no monochromatic k -term progression. The large claims below instead use a *succinct Rabung certificate* (p, r, k) : an autonomous checker re-establishes the hypotheses of a proved construction theorem. This is not an exhaustive enumeration of all progressions in the resulting colouring.

Let p be a prime with $p \equiv 1 \pmod{r}$ and $p > k$, let g be a primitive root modulo p , and let $\text{ind}(x)$ denote the discrete logarithm of x to base g . The *power-residue coloring* is

$$c(x) = \text{ind}(x) \bmod r, \quad x \in \{1, \dots, p-1\},$$

i.e. the partition of $\mathbb{Z}_p^*$ into the r classes of r -th power residues. Its key property is multiplicativity: $c(xy) = c(x) + c(y) \pmod{r}$. Consequently, dilating a progression $\{a, a+d, \dots, a+(k-1)d\}$ with $p \nmid d$ by $u = d^{-1} \bmod p$ shifts all colors by the constant $c(u)$ and maps the progression to k consecutive residues modulo p . The search for monochromatic progressions therefore reduces to a linear scan of the run lengths of the sequence $c(1), c(2), \dots, c(p-1)$, plus a boundary condition at the multiples of p ; this reduction from an $O(p^2/k)$ test to an $O(p)$ test per prime is what makes a mass scan possible.

2.2 Rabung's theorem and the operational criterion

Rabung [11] proved that the extension ϑ' of the power-residue coloring to $[0, (k-1)p]$, with the multiples of p colored by any non-constant assignment, is free of monochromatic k -term progressions *if and only if*

- (a) no k consecutive positions in $[1, p-1]$ receive the same color ($\text{max run} < k$); and
- (b) a boundary condition holds: write $\vartheta(-1) = +1$ if $p-1$ lies in the class of the r -th powers (i.e. $c(p-1) = 0$; the *palindromic* case — additively, in the appendix, $\vartheta(-1) = +1$ corresponds to $c(p-1) = 0$ and $\vartheta(-1) = -1$ to $c(p-1) = r/2$) and $\vartheta(-1) = -1$ otherwise. If $\vartheta(-1) = +1$ the integers $1, 2, \dots, \lceil (k-1)/2 \rceil$ must not all lie in the same class, while if $\vartheta(-1) = -1$ the same is required of $1, 2, \dots, k-1$. A *singleton* counts as “all in the same class”, and so fails (b).

The second alternative $c(p-1) = r/2$ can occur only for even r . If r is odd, $(p-1)/r$ is even and hence $c(-1) = 0$ necessarily.

Remark 1. Both details of (b) matter. Replacing the ceiling by a floor, or treating a singleton as vacuously satisfying the condition, yields a criterion that is *not* equivalent to Rabung's. Explicit counterexamples occur already at $p = 13$, $r = 2$, $k = 4$ for the floor variant and at $p = 5$, $r = 2$, $k = 3$ when a singleton is mishandled. The ceiling and floor coincide only for odd k . We record this because an early version of our own tester made exactly these two mistakes, and comparison against the oracle described below is what caught them.

Since $[0, (k-1)p]$ contains $(k-1)p + 1$ integers, a prime satisfying (a) $\wedge$ (b) yields

$$W(r, k) > (k-1)p + 1. \tag{2}$$

We call such a prime *valid* for (r, k) .

Our implementations test an equivalent form of (b), denoted (B^*) , which is uniform in the parity of k and the value of $\vartheta(-1)$: for every shift $j \in \{0, \dots, k-1\}$, the $k-1$ character positions $\{-j, \dots, -1, 1, \dots, k-1-j\}$ of a k -window centered on a multiple of p must not all receive the same color. The equivalence rests on the multiplicative structure: if a single shifted window is monochromatic, its dilates realize every color at the central position (surjectivity of c requires exactly $p \equiv 1 \pmod{r}$), so the joker position cannot be colored at all; otherwise any non-constant coloring of the multiples of p works. A literal transcription of (b) is error-prone — an early version of our own tester mishandled the singleton case and the rounding of $(k-1)/2$ — which is one reason we insist on machine verification of the criterion itself.

2.3 Machine verification of the criterion

We verified the criterion $(a) \wedge (B^*)$ against a frozen reference oracle that explicitly builds the coloring of $[1, (k-1)p+1]$ and exhaustively checks all k -term progressions. One bookkeeping detail matters: Rabung’s construction lives on $[0, (k-1)p]$, and the oracle works on its translate by one—it colors x by the class of $(x-1) \pmod{p}$ and places the jokers at $x \equiv 1 \pmod{p}$ —so the two objects are literally the same coloring up to relabeling of positions, and freedom from monochromatic progressions is translation-invariant. A sweep over all primes $p < 10^4$ with $p \equiv 1 \pmod{r}$, $p > k$, $r \in [2, 6]$, $k \in [3, 8]$ — 20 170 cases, of which 4 499 were valid according to the criterion — reported zero disagreements. A fresh staging replay reproduced those counts and its exact source identity, environment, and output streams are archived with this repository. Because that replay was not made from the final clean release candidate, the final CPU gate must still rerun it from the exact candidate commit and preserve durable raw outputs before the local tag is published. A stronger test enumerates all r^k colorings of the joker positions ($p < 150$, $r \leq 4$, $k \leq 5$) and confirms both directions of the equivalence: $(a) \wedge (b)$ holds if and only if some valid extension exists, and when it holds every non-constant joker coloring is valid. We emphasize the epistemic status: Rabung’s theorem is proved in [11]; the agreement of our operational form with the explicit oracle is exhaustively machine-checked on the stated domain.

2.4 Certificates and verification regimes

For the bounds announced in Section 4 the certificate length $(k-1)p+1$ is of order 3×10^{10} to 10^{11} , so running the naive quadratic oracle on the explicit certificate is not feasible. Our verification policy therefore distinguishes two regimes. On the finite validation domain actually reported above ($p < 10^4$, $k \leq 8$, hence certificate length below 7×10^4), the explicit certificate was checked directly by the reference oracle. We make no general feasibility claim for length 10^7 : a successful naive oracle must inspect

$$\sum_{d=1}^{\lfloor (n-1)/(k-1) \rfloor} (n - (k-1)d)$$

progressions. For the large primes, the proof of (2) is Rabung’s theorem together with conditions $(a) \wedge (b)$, and *these* are what we verify redundantly: the mirror, full-interval, and CPU-walk paths, followed by separate CPU implementations of the full criterion. Section 3.2 records which paths share source code or arithmetic; cross-agreement is not described as statistical or implementation independence.

Two kinds of certificate should be distinguished. An *explicit* certificate is a full coloring of $\{1, \dots, n\}$; for the bounds of Section 4, with n up to 10^{11} , distributing one is neither feasible nor useful. What the reproducibility package distributes is a *succinct Rabung certificate*: the triple (p, r, k) , from which a stand-alone independent verifier, `verify_claim`, re-derives everything. Its protocol, in full:

1. check that p is prime (deterministic Miller–Rabin on the first twelve prime bases $2, 3, 5, \dots, 37$; Sorenson and Webster’s Theorem 1.1 gives $\psi_{12} = 318\,665\,857\,834\,031\,151\,167\,461 > 2^{64}$, so this is exact through the unsigned 64-bit range [12], far beyond the $p < 2^{32}$ actually used);
2. check the preconditions $p \equiv 1 \pmod{r}$ and $p > k$;
3. *find* a primitive root g by trial, certifying primitivity against the full trial-division factorization of $p - 1$ (nothing about g is taken as input);
4. build the residue classes by modular exponentiation— $x^{(p-1)/r}$ compared against the powers of $\zeta = g^{(p-1)/r}$, a method independent of both the discrete-logarithm walk and the GPU pipeline;
5. test $(a) \wedge (B^*)$ and print ACCEPT or REJECT.

The whole run takes $O(p \log p)$ time and $O(p)$ memory. Each claimed bound in this paper can thus be re-derived from the triple (p, r, k) alone—no generator, witness, or auxiliary data are part of the certificate—in about two to just over seven minutes on a single core (Table 1); the certificates are independently verifiable in this precise, succinct sense.

p	(r, k)	wall time	peak RAM	verdict
1 138 900 957	(2, 25)	130 s	1.1 GB	ACCEPT
1 961 601 427	(3, 17)	229 s	1.9 GB	ACCEPT
3 459 826 103	(2, 27)	404 s	3.3 GB	ACCEPT
3 476 732 783	(2, 28)	425 s	3.3 GB	ACCEPT

Table 1: Measured cost of the independent verifier on the four headline certificates. The two high-range rows are cross-documented with the corrected-scanner and Montgomery-free checks required by the fail-closed claim manifest protocol. Those release manifests are not yet populated, so these historical timings are not themselves release evidence. Single-threaded on an AMD Ryzen 7 9800X3D under Ubuntu 26.04, `g++ 15.2.0 -O2 -std=c++17`. The time is modest; the memory is the real constraint, since the present implementation stores the colouring, i.e. $O(p)$ bytes. A two-pass streaming variant with $O(1)$ memory is possible and would be a worthwhile improvement, but it is not what we ran, and we report what we ran.

3 The GPU campaign

3.1 Fused character evaluation

The scan kernel exploits the fact that the additive order of positions is free: we iterate $x = 1, 2, \dots, p - 1$ directly and evaluate the color of x on the fly by modular exponentiation, fused with the run-length bookkeeping. Nothing is sorted and no per-element value is materialized

in memory. A single pass evaluates $y = x^{(p-1)/m}$ with $m = \text{lcm}\{r \leq 9 : r \mid p-1\} \leq 2520$; a table lookup of y among the m -th roots of unity gives $\text{ind}(x) \bmod m$, hence the color for every modulus $r \leq 9$ simultaneously. Runs are summarized per segment by (prefix, suffix, best interior) triples combined with an associative, order-respecting reduction.

A second symmetry halves the work. Since $c(p-x) = c(x) + c(-1)$, the second half of the sequence is the reversal of the first half shifted by the constant $c(-1)$; it suffices to scan $x = 1, \dots, (p-1)/2$ and to splice the two halves at the center. We refer to this half-interval variant as the mirror scan.

3.2 Validation by redundant and independent paths

Correctness is supported by cross-agreement among redundant paths and by separate-source checkers rather than by trust in any single path. On all primes $p < 105\,000$ (both residue classes of $p \bmod 4$ and varied m ; the r -range of each comparison set is given with it):

- the sort-based GPU pipeline (an earlier, algorithmically distinct GPU design retained as a redundant path) agrees with the CPU reference on 50 105 comparisons ($r \in [2, 6]$) with zero disagreements;
- the fused multi- r kernel agrees with the CPU reference on 27 508 comparisons ($r \in [2, 9]$) with zero disagreements (and the specialized $r = 2, 3$ variant on a further 9 068);
- the mirror scan, the fused full scan, the sort-based pipeline, and the CPU reference agree on 33 327 comparisons over 10^4 primes with zero disagreements.

These comparison totals and the performance measurements below are historical pre-flight records. Their exact raw GPU outputs and benchmark logs must still be recovered from the campaign PC and bound to the release candidate; the portable staging suite exercises the algorithms but does not treat these totals as release-qualified evidence. For $r = 2$ a CPU Jacobi-symbol evaluation provides one further independent route. This redundancy is not decorative: during pre-flight testing the cross-check caught a genuine bug (a non-commutative run-summary operator fed to a reduction that assumes commutativity, producing phantom runs at $p \sim 1.8 \times 10^9$); the repaired, order-respecting reduction is covered by a regression test that has run at every commit since.

The word *independent* is reserved below for separations of source and arithmetic. The main paths have the following dependency structure.

3.3 Measured throughput

On the target hardware (one consumer RTX 5090, CUDA, 32-bit Montgomery arithmetic) the mirror multi- r kernel was benchmarked at 4.86×10^{10} physical evaluations of x per second at $p \approx 1.2 \times 10^9$. Mirror reconstruction makes this equivalent to 9.72×10^{10} positions of the full sequence per second. The same run performed 2.09×10^{12} modular multiplications per second, 61% of the measured pure-multiplication plateau (3.40×10^{12}). Under the 431 W campaign power cap the observed kernel throughput was $4.2\text{--}4.45 \times 10^{10}$ physical evaluations, or $8.4\text{--}8.9 \times 10^{10}$ reconstructed full-sequence positions, per second. These are active-kernel rates. The ten-day campaign duration is wall time and also includes sieving, host work, checkpointing, cross-checks, and restarts; we do not quote a wall-clock modular-multiplication rate. Several plausible optimizations (e.g. a 2-adic factorization of the exponentiation) were implemented, measured, and rejected as memory-bound regressions.

path	language	modular arithmetic	class/run construction	shared components
mirror V2a	CUDA C++	GPU Montgomery	character, half interval	same source and helpers as full B7
full B7	CUDA C++	GPU Montgomery	character, full interval	same source and helpers as V2a
sort-based GPU	CUDA C++	GPU Montgomery	sort then run scan	earlier GPU design; arithmetic is not independent
CPU walk	C++ host	native 64-bit walk	discrete logarithm, sequential	same executable and host primitive-root code
CPU criterion	C++	separate 128-bit path	separate walk and (B*)	separate source and binary
verify_claim	C++	128-bit exponentiation	roots of unity and full criterion	strongest source/arithmetic separation
high-range witness	C	no Montgomery	full sequential walk	separate arithmetic; tests (a), not (B*)

Table 2: Dependency matrix for the verification paths. V2a and B7 are redundant paths, not independent implementations. The principal independent checks are the separate CPU criterion, **verify_claim**, and, for the high-range arithmetic defect, the Montgomery-free witness.

3.4 Campaign execution

According to the preserved campaign design and summaries, the production run was configured to scan every prime in $[9.70 \times 10^8, 2.0 \times 10^9]$ (each prime tested for every $r \leq 9$ dividing $p - 1$), starting just above the largest published prime 969 347 371 of Monroe’s distributed scan [7], in a single multi- r pass ($r \leq 9$). (The narrow interval $[969\,347\,371, 9.70 \times 10^8]$ was therefore not scanned; it is immaterial, since any valid prime in it would give a bound below every one we report.) The historical checkpoint reports that the range was processed in 515 restartable chunks of width 2×10^6 , each with a fixed walltime cap. The registered revision map spans *eight* git revisions and *four* scanner-source fingerprints. Their exact allocation and the scientific effect of each transition are generated from the audit registry below. The (3, 17) max-run statistic and its target predicate are unchanged across all four fingerprints. The first functional expansion added only (2, 26)–(2, 28), whose early online tallies are therefore incomplete; the second added congruence counters and reproduced the previous target profiles and checksums in an old/new comparison. This supports code-path comparability for the retained (3, 17) counts, although version-stratified dispersion diagnostics remain distinct from that code audit. Each checkpoint row records the range, duration, git revision, and deterministic scientific checksum. The revised orchestrator additionally binds a new checkpoint to the exact half-open range, chunk width, target set, criterion, and logical output directory; completed keys must form a canonical prefix, and smoke tests use a separate output directory. This post-campaign safeguard does not retrofit the historical checkpoint, whose immutable release copy remains an audit gate. Because the host memory is not error-correcting, the historical journal reports that every 50 chunks an automatic cross-check re-verified a sample of primes with the redundant scan paths and separate CPU checks (nine completed in full agreement; a tenth attempt timed out and was not counted). The raw outputs remain a release artifact, and these samples do not turn the single production campaign into a duplicated run. Separate historical point checks later reported acceptance of the claim primes (Section 4), while the complete checkpoint, chunk, and ordered-prime evidence remains subject to the release gates below. The registry binds every row to full Git commit

source hash	chunks	revisions (number of chunks)	targets	change and transition check
87f10166	1–7	ebd54ea (7)	10	Initial campaign fingerprint; fused and mirror max-run paths already active. Covered by the frozen pre-flight agreement suite.
3598aea6	8–35	657a140 (10), 185b868 (18)	10	Added the point-verification mode and record reporting; the production target predicate was unchanged. Point claims were checked by mirror, full, CPU-walk, and separate verifiers.
cca301fe	36–78	062cc3c (43)	13	Added the three two-colour bonus targets; the already-computed $r=2$ max-run profile was reused. The new bookkeeping was exercised on [1026949000,1026950000]; the (3,17) path was unchanged.
3b5d1049	79–515	4f9e2cf (1), c3a9b9e (274), a93cec9 (72), 7c1ba34 (90)	13	Added aggregate congruence counters; later revisions changed the driver or theory files, not scan_gpu.cu. Old/new TARGET and CHECKSUM outputs agreed exactly and an end-to-end test passed.

Table 3: Scanner-source fingerprints and Git revisions used by the 515 campaign chunks. The registry also records the active target list and Git blob identity.

and Git-blob identities. The exact raw transition-test outputs and their hashes remain required release artifacts; the prose summary in Table 3 is not a substitute for those files.

Exhaustiveness of the prime enumeration. The per-prime computation is validated by the redundancy above; the *enumeration* of primes deserves its own audit, since the density claims of Section 5 are claims about *all* primes in the window. The scanner enumerates primes with a plain in-process sieve per chunk; chunks are contiguous by construction ($[lo + 2i \cdot 10^6, lo + 2(i+1) \cdot 10^6)$), and the checkpoint protocol resumes only at chunk boundaries, so interruptions can neither skip nor duplicate primes within the range. We verified the retained aggregate coverage counts against a separately written segmented sieve with no shared code: over the 498 chunks for which per-chunk histograms were logged (the histogram feature was added at an early restart, from $p = 1.004 \times 10^9$ on), the campaign’s per-chunk counts of primes in every congruence class $p \equiv 1 \pmod{r}$, $2 \leq r \leq 9$ —3984 counts in all—agree with the independent sieve *exactly*, with zero discrepancies (47181875 primes on those chunks). For the first 17 chunks, which predate the histogram feature, per-class counts were not logged during the campaign; that range, $[9.70 \times 10^8, 1.004 \times 10^9)$, was therefore re-scanned after the campaign in one aggregate invocation with the post-campaign audit build (commit c81383885bd9; full identities are in the registry). The eight independent counts were committed before that invocation completed (commit 30e8b83df3c0). A version-specific, fail-closed parser now checks the preserved transcript, whose SHA-256 is dff4e409...de184; it verifies every historical TARGET, candidate, COLQ, HISTO, and TOP row before comparing the eight histogram sums with those precommitted counts.

The re-scan also lists exactly 405 run-valid (3, 17) candidates, consistently with its TARGET row and its $r = 3$ histogram. It was one invocation over the whole interval, however, so it supplies aggregate class totals rather than 17×8 per-chunk cells. These audits establish exact agreement

r	$\#\{p : p \equiv 1 \pmod{r}\}$	r	$\#\{p : p \equiv 1 \pmod{r}\}$
2	1 641 614	6	821 005
3	821 005	7	273 695
4	820 559	8	410 485
5	410 333	9	273 955

Table 4: Aggregate congruence-class totals from the post-campaign re-scan of the first 17 chunks. All eight equal the independent counts committed before the re-scan completed. This is one aggregate comparison, not 17 per-chunk comparisons and not an ordered-prime identity test.

of the counts; counts alone do not establish identity of the ordered prime lists. The audit utility therefore also emits a canonical ordered stream for byte comparison and hashing. Populating its per-chunk identity manifest for all 515 chunks is a release gate, not a consequence inferred from the 3 984 matching counts. The four historical scanner blobs contain a byte-identical 249-byte `sieve_primes` function (SHA-256 recorded in the registry). That function accepted an inclusive upper endpoint, whereas the audit stream is half-open; every campaign boundary is an even integer greater than two, hence composite, so the two conventions enumerate exactly the same primes on all 515 chunks. The current scanner makes the half-open convention explicit. In total the window $[9.7 \times 10^8, 2 \times 10^9)$ contains 48 823 489 primes in the independent sieve. Across the retained 498-chunk histograms and the aggregate 17-chunk re-scan, the preserved outputs report corresponding per-class totals; full byte-level ordered-prime identity remains a release gate. The independent sieve counts 1 448 736 primes in $[9.7 \times 10^8, 10^9)$, hence 47 374 753 in $[10^9, 2 \times 10^9)$. This equals the externally published difference $\pi(2 \times 10^9) - \pi(10^9) = 98\,222\,287 - 50\,847\,534$ from Oliveira e Silva’s table [9]; it checks the interval difference, not the two absolute values independently.

The historical journal reports a ten-day campaign, a clean chunk-boundary stop and checkpoint resume after one disk-full incident caused by an unrelated job, and an intact checkpoint followed by resume after one forced reboot. The available summaries, deterministic chunk checksums, periodic cross-checks, and aggregate coverage comparison report no inconsistency. They do not yet prove the stronger phrase “without data loss”: the release recovery must still substantiate checkpoint continuity, the chunk 7 reconstruction, and the raw artifact inventory. These checks hash summaries, not every byte of output. Candidate direct Rabung champions detected during the scan were frozen and passed to the verification protocol of Section 4; no bound was recorded from the scanning pass alone.

4 Certified direct claims and closed lower bounds

4.1 Verification protocol

The computational source of truth contains exactly thirteen direct Rabung claims: nine three-colour claims for $17 \leq k \leq 25$ and four two-colour claims for $25 \leq k \leq 28$. For each triple (p, r, k) the release protocol requires:

1. two executions of the structured triplet {mirror scan, full scan, CPU walk}, with internal profile equality and equality between executions;
2. two acceptances by the separate full CPU criterion;

3. acceptance by the stand-alone `verify_claim`; and
4. for the two primes above the historical Montgomery-safety threshold, an additional full-interval witness that uses no Montgomery arithmetic.

The parser is fail-closed: it checks the process status, the exact p, r, k , all profile equalities, (a), (B*), and the final verdict, and rejects any output containing `DIFF`, `FAIL`, `ECHEC`, or a negative verdict. Its digest engages the claim identity, profiles, criterion values, and final verdict. The enclosing JSON manifest separately records the source and binary hashes, compiler version, exact commands, and hashes of every raw output. A SHA-256 of that manifest therefore binds the computational claim to its build and evidence. The final Markdown and $\text{\LaTeX}$ claim tables are generated only from an exact set of thirteen accepted manifests.

Validated-claim table unavailable in this build. The canonical table appears only when the exact set of thirteen fail-closed claim manifests is available and validated. The publication contract requires the generated table in a release build.

This rule is stricter than the historical campaign transcript. The latter recorded a max-run checksum that did not engage p , and some pasted `verify_claim` outputs were truncated. Those files are retained as historical evidence, not used as the final source of truth. The thirteen claims are documented in `audit/claims.json`; the expensive final point-verification run must populate the complete raw manifests before the release is frozen. The numerical bounds in this section use only the thirteen triples, each of which is also re-derived by the separate-source checker described in Section 2.4.

4.2 Three colours: direct and recurrence-derived bounds

The historical scan produced nine candidate direct Rabung certificates. The headline value

$$W(3, 17) > 31\,385\,622\,833$$

comes from $p = 1\,961\,601\,427$ and improves Monroe’s published $15\,509\,557\,937$ by a factor 2.02 [7]. A direct certificate is not automatically a general record: published bounds in other colour columns may imply a stronger result through a recurrence.

Let q_k denote the largest prime not exceeding k . From Blankenship–Cummings–Taranchuk [1, Theorem 2.1],

$$W(3, k) > q_k(W(2, k) - 1).$$

If the registered input is the strict integer bound $W(2, k) > B$, then $W(2, k) - 1 \geq B$, so the generated consequence is exactly

$$W(3, k) > q_k B, \tag{3}$$

with no off-by-one ambiguity. The baseline program iterates this rule and Xu’s recurrence to a fixed point, retains a provenance path for every generated candidate, and selects the best bound only afterwards. Its output also embeds the source catalogue and SHA-256 hashes of both input registries.

Thus the direct certificates for $17 \leq k \leq 21$ remain the strongest bounds located in the registered published-and-archived source corpus after recurrence closure. For $22 \leq k \leq 25$ the direct rows remain improvements within the direct Rabung family but are not best general bounds. In particular, the published two-colour values already dominate the direct rows at

k	direct p	direct bound	best closed bound	method
17	1 961 601 427	31 385 622 833	31 385 622 833	direct Rabung
18	1 999 994 851	33 999 912 468	33 999 912 468	direct Rabung
19	1 999 999 927	35 999 998 687	35 999 998 687	direct Rabung
20	1 999 999 927	37 999 998 614	37 999 998 614	direct Rabung
21	1 999 999 927	39 999 998 541	39 999 998 541	direct Rabung
22	1 999 999 927	41 999 998 468	49 058 715 046	BCT, $q = 19$ (published $W(2, k)$)
23	1 999 999 927	43 999 998 395	142 741 265 701	BCT, $q = 23$ (published $W(2, k)$)
24	1 999 999 927	45 999 998 322	252 542 143 876	BCT, $q = 23$ (published $W(2, k)$)
25	1 999 999 927	47 999 998 249	628 673 328 287	BCT, $q = 23$ (archived input)
26	–	–	1 221 912 599 848	BCT, $q = 23$ (archived input)
27	–	–	2 068 976 009 617	BCT, $q = 23$ (archived input)
28	–	–	2 159 051 058 266	BCT, $q = 23$ (archived input)

Table 5: Direct three-colour Rabung certificates and the best general lower bounds obtained after closing the registered baselines under the published Blankenship–Cummings–Taranchuk and Xu recurrences. A dash means that the GPU campaign made no direct three-colour claim for that length. For $k = 22, 23, 24, 25$ the direct certificates are new champions within the direct Rabung family but are not general records. Every entry is generated together with a machine-readable provenance path in `audit/generated/bounds_closure.json`.

$k = 22, 23, 24$. For $k = 25$, even Monroe’s published $W(2, 25) > 23\,003\,662\,489$ would give $W(3, 25) > 529\,084\,237\,247$; the archived input accepted by historical independent checks, $W(2, 25) > 27\,333\,622\,969$ strengthens this to the first line below.

$$\begin{aligned}
W(3, 25) &> 628\,673\,328\,287 \\
W(3, 26) &> 1\,221\,912\,599\,848 \\
W(3, 27) &> 2\,068\,976\,009\,617 \\
W(3, 28) &> 2\,159\,051\,058\,266
\end{aligned}$$

The four displayed inequalities are theorem consequences, not additional GPU claims. Priority for the two-colour primes remains Monroe’s; we report the candidate re-verification and apply the published recurrence. The final reproducibility claim remains conditional on completion of the raw-manifest release protocol above. Xu’s recurrence is included in the automatic closure but does not improve any of these three-colour winners.

4.3 Two colours and archived provenance

After the scan, we located a table from a second, unpublished phase of Monroe’s BOINC project (2021, two colours only) [6]. The author’s project page states that this phase pushed the checked range from one billion to four billion [8]; the largest prime actually listed in the recovered table is 3 478 685 143. We credit the listed primes to Monroe and use our checker only to establish that the four triples below satisfy Rabung’s criterion.

For $k = 25$ the historical campaign summaries identify the same last candidate valid prime in $[9.7 \times 10^8, 2 \times 10^9)$ as the archived table; complete campaign recovery and ordered-prime identity remain release gates. For $k = 26, 27, 28$ our campaign-range candidates are weaker; the displayed archive primes were checked directly in historical runs. Those runs supply candidate evidence for the listed triples; final release manifests must make the checks independently auditable.

k	prime p	$W(2, k) >$	method and provenance
25	1 138 900 957	27 333 622 969	archive and historical recheck; final manifest pending
26	2 125 065 391	53 126 634 776	archived Monroe prime; final manifest pending
27	3 459 826 103	89 955 478 679	historical no-Montgomery recheck; final manifest pending
28	3 476 732 783	93 871 785 142	historical no-Montgomery recheck; final manifest pending

Table 6: Two-colour succinct Rabung certificates used as recurrence inputs. The method column separates priority for the archived prime from the candidate re-verification; final raw manifests are still pending.

The computation does not establish that Monroe’s historical phase tested every prime up to four billion: that coverage statement is an attributed archival claim, not a consequence of the recovered table.

The source is the Internet Archive capture of 28 April 2022 of <http://vdownumbers.org/bounds.txt> [6]; its own header reads “Last updated: 18 Aug 2021”. The release records the immutable capture URL, both dates, the source-repository blob identity, and its SHA-256 in `publication/baseline-provenance.json`; it does not redistribute the third-party table absent permission. Once the thirteen fail-closed manifests and their raw outputs are populated, the stand-alone verifier will establish a separately auditable verification event. The defensible novelty formulation for that release is therefore: an independently reproduced verification accompanied by a stand-alone public checker and complete reproducible logs, not first discovery of the primes.

5 An empirical density comparison

The campaign classified each tested prime by the run-valid predicate and preserved the survivor event list and aggregate counts, but not the complete per-prime max-run profile. This section uses those retained data for a deliberately narrow purpose: a prospective forecast comparison for the run-valid $(3, 17)$ primes beyond a fixed training cut. It does not promote the comparison to a theorem, a universal density law, or a proof that any Rabung search has been exhausted. All probabilistic statements below are conditional on the explicitly stated count model.

5.1 Population and mirror reference

Call a prime *run-valid* for (r, k) when condition (a) holds, that is, when $\text{maxrun}(p, r) < k$, and *valid* when $(a) \wedge (b)$ holds. The statistical estimand in this section is the run-valid population. The bounds of Section 4, by contrast, use the full criterion. Condition (b) depends on k and is not represented in the versioned run-valid event list used below; it must therefore not be silently absorbed into a common density correction.

The coloring satisfies

$$c(p - x) = c(x) + c(-1) \pmod{r}.$$

Consequently only the first $(p - 1)/2$ positions are free. The natural i.i.d. reference for the maximal run therefore has length $n = (p - 1)/2$, not $p - 1$. Descriptively, this accounts for most of the shift in the natural reference length. We make no numerical mean-run claim here, because the versioned event list records survivors rather than the full per-prime max-run profile.

For $r = 3$ the case $c(-1) = r/2$ cannot occur, since r is odd; more generally that case exists only for even r .

5.2 Bare reference and domain

With $(p - 1)/2$ free positions, the elementary Poisson-clumping reference is

$$\Lambda_{r,k}(p) = \frac{(r - 1)(p - 1)}{2r^k}, \quad g_0(p) = e^{-\Lambda_{r,k}(p)}.$$

It motivates the qualitative scale $p \asymp kr^k$ at which direct Rabung survivors become rare. We do not use the local fit below to quote precise numbers of future primes, to declare a well exhausted, or to assign narrow uncertainty intervals far outside the fitted window.

The program frozen during the campaign used the operational convention

$$\tilde{\Lambda}_{r,k}(p) = \frac{(r - 1)p}{2r^k}, \quad \tilde{g}_0(p) = e^{-\tilde{\Lambda}_{r,k}(p)}.$$

Thus $\tilde{\Lambda} - \Lambda = (r - 1)/(2r^k)$. We retain the tilded convention below so that the holdout calculation is exactly the one that was frozen, while keeping Λ as the natural half-length reference.

Remark 2 (Domain of the empirical comparison). The two corrected candidates considered below are

$$\tilde{g}_1(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + me^{-\beta\tilde{\Lambda}_{3,17}(p)}, \quad \tilde{g}_2(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + \eta,$$

with $m = 8 \cdot 10^{-4}$, $\beta = 1/2$, and $\eta = 1.62 \cdot 10^{-5}$. These are local intensity models for condition (a) in the observed deep-tail window. In particular $\tilde{g}_1(p) \rightarrow 1 + m > 1$ as $\tilde{\Lambda} \rightarrow 0$, so it is not a global probability law. The numerical parameters were selected on the training window; freezing them prevents holdout tuning but does not remove their estimation uncertainty.

5.3 Prospective forecast comparison

After completion of chunk 180, when 1727 run-valid $(3, 17)$ primes had been observed, the three numerical candidates $\tilde{g}_0, \tilde{g}_1, \tilde{g}_2$ were fixed. The four scheduled checkpoint values 250, 300, 400, 515 were recorded before the first read. The versioned 1859-entry list, labelled complete by the historical analysis, reconstructs the count in every campaign chunk conditional on that list.

Let

$$I_j = [970\,000\,000 + (j - 1)2\,000\,000, 970\,000\,000 + j2\,000\,000)$$

and, for a set A of chunks, define the corrected mean derived from a frozen intensity,

$$\mu_\ell(A) = \sum_{j \in A} \int_{I_j} \frac{\tilde{g}_\ell(p)}{2 \log p} dp, \quad \ell \in \{0, 1, 2\}.$$

The factor $1/(2 \log p)$ is the analytic density used for primes $p \equiv 1 \pmod{3}$. Table 7 uses the four *disjoint* increments as its primary view.

The corresponding cumulative observations are 90, 110, 126, 132. They are four views of the same trajectory, not four replications. The fixed decaying forecast is close to the observed total, the frozen constant-floor candidate substantially overpredicts it, and the bare reference underpredicts it.

Remark 3 (Frozen readout and post hoc diagnostics). Three layers must be kept distinct.

First, the frozen inputs were the three intensity functions, their numerical parameters, the baseline count, and—before the first scheduled read—the reading times. Second, the live tool

Table 7: Prospectively scheduled (3, 17) readings, reported as disjoint increments. The model columns are means integrated over the full half-open chunk intervals.

chunks	observed	bare $\tilde{g}_0$	decaying $\tilde{g}_1$	constant floor $\tilde{g}_2$
181–250	90	68.35	80.26	122.20
251–300	20	18.75	24.04	57.06
301–400	16	12.58	18.59	88.87
401–515	6	2.81	5.82	90.04
total	132	102.49	128.72	358.17

selected the closest displayed mean and printed Wald-type discrepancies $(N - \mu)/\sqrt{N}$ for the bare and constant-floor candidates. That was the original prospective readout. It did not compute deviance residuals or exact Poisson tails; two manually noted standardizations for the decaying candidate were later found to use the wrong denominator.

Third, all deviance, dispersion, autocorrelation, negative-binomial, and block calculations are retrospective diagnostics. The archival script also used chunk midpoints as the two endpoints of a cumulative integral. Table 7 corrects that endpoint convention by integrating the full observed chunk intervals. This changes no observation and no frozen operational intensity function; it changes only the integration limits. The corrected numerical means are therefore post-campaign recalculations of the frozen functions. Replacing the operational $\tilde{\Lambda} = p/3^{17}$ by the natural $\Lambda = (p - 1)/3^{17}$ changes every holdout total by less than $9 \cdot 10^{-7}$ and changes no cell of Table 7 at two decimal places.

The four increments are non-overlapping. They are independent only under the specified inhomogeneous-Poisson model; disjointness alone is not evidence of independence. Accordingly, we do not promote a conditional Poisson tail probability as a model-robust significance level.

The versioned event list permits limited retrospective checks. On the 335 post-freeze chunks, the frozen decaying model gives Pearson statistic $X^2 = 324.2$ (quasi-Poisson scale $X^2/335 = 0.968$); an NB2 sensitivity fit puts the additional-dispersion parameter at the Poisson boundary, and the Poisson deviance is 210.46. The Pearson residuals have Ljung–Box $Q_{12} = 7.17$ (asymptotic conditional reference $p = 0.85$), and aggregation into blocks of 5, 10, 20 chunks gives Pearson scales 0.81, 0.96, 1.05. These checks reveal no gross overdispersion or serial correlation for that fixed model, but they remain post hoc and have limited power in a sparse, non-stationary tail. A moving-block resampling of the per-chunk log-score difference (circular blocks of length 10, 20 000 seeded replicates) gives a mean decaying-minus-bare advantage of 0.01185 per chunk, with percentile interval $[-0.00509, 0.02957]$ and bootstrap fraction 0.092 at or below zero. For decaying minus the frozen constant-floor model, the corresponding interval is $[0.31298, 0.49363]$ and no replicate is non-positive. Thus the former comparison is less decisive than the latter. We therefore report the forecast ordering, not a universal fitted family or a discovery-level p -value.

The scanner changed earlier in the campaign, but the final source fingerprint begins at chunk 79. Thus all chunks after the forecast cut use the same scanner fingerprint. This removes scanner-source changes as a confounder for the prospective comparison, while whole-campaign descriptive aggregates still require the revision table discussed in Section 3.4.

5.4 Summary

The mirror symmetry supplies a useful first-order reference, and one fixed decaying correction tracked the held-out (3, 17) trajectory more closely than the bare and constant-floor candidates. The result is an empirically useful forecast check for one target and one finite window. It is not four independent confirmations, not a proof of a universal two-exponential law, and not a quantitative certificate that any future Rabung search is empty.

6 Prospective and retrospective analyses

The computation mixes proved claims, validated certificates, descriptive measurements, and statistical models. We keep those categories separate: the lower bounds are consequences of the Rabung theorem after machine verification of its hypotheses, whereas the density discussion of Section 5 is empirical and model-conditional.

What was fixed prospectively. Chunks 1–180 form the training window for the (3, 17) comparison. At chunk 180 the repository fixed the baseline count and the three numerical intensities of Remark 2. A later commit, still before chunk 250, fixed the reading schedule 250, 300, 400, 515. No parameter was changed after the cut. The word *prospective* refers only to the data after chunk 180; the discovery of the candidate forms and the choice of their parameters used the training data and are not called blind.

The original rule. The live program compared the cumulative observed increment with the three means displayed by the frozen functions, identified the closest displayed mean, and printed Wald-type discrepancies normalized by $\sqrt{N_{\text{obs}}}$ for two candidates. This is the rule that was actually executed at the scheduled reads. It was not an exact Poisson test and did not contain the deviance or dispersion analyses now reported in the paper. Two manually transcribed standardizations for the third candidate were erroneous; the counts and forecast functions were not.

Retrospective corrections and sensitivities. After completion of the campaign we recomputed the means on the full half-open chunk intervals, rather than between chunk midpoints, and calculated Pearson and deviance diagnostics, residual autocorrelations, a quasi-Poisson scale, an NB2 sensitivity fit, and block aggregations. These are labelled post hoc. They are useful checks on the specified fixed forecast, but they do not retroactively become part of the prospective decision rule. Likewise, freezing m and β prevents holdout refitting but does not make their training-window uncertainty vanish; we therefore make no family-wide or long-range extrapolative claim from this comparison.

One trajectory, four scheduled views. The four cumulative readings share their early data. The primary presentation is therefore the four disjoint increments of Table 7. Even those increments are statistically independent only conditional on the inhomogeneous-Poisson model. We retain the cumulative values as an audit trail, not as four replications, and do not advertise a small conditional tail as a model-robust significance level.

Scanner revisions. The 515 campaign chunks span eight git revisions and four scanner-source fingerprints. The final fingerprint starts at chunk 79, before the training cut, so the complete post-freeze comparison uses one scanner source. This is the relevant stratification fact for Section 5.3. Descriptive whole-campaign summaries must nevertheless be accompanied by the full revision table and by tests across the earlier transitions.

Targeted independent checks. The historical campaign journal reports that 200 sampled run-valid (3, 17) primes were re-tested through a separate CPU discrete-logarithm walk without a reported failure. The present event-list audit does not reproduce that test; its sampled-prime list and complete raw output must be included and hashed in the release before the numerical statement is treated as machine-audited. Even then, the check addresses false survivors, not omissions, prime enumeration, dispersion, or temporal dependence.

Reproducibility boundary. The versioned 1859-entry list of reported run-valid (3, 17) primes is sufficient to reconstruct all 515 success counts and the disjoint increments in Table 7, conditional on that list. It does not by itself re-evaluate the max-run predicate or prove that no run-valid prime was omitted. The live tracking script additionally reads `results/campaign/checkpoint.json`, while the campaign directory is excluded from ordinary source tracking. An end-to-end archival claim therefore requires an immutable copy of the checkpoint, the raw per-chunk outputs, and a machine-readable revision map in the final data release. Until those files are part of the cited release, the git history alone reproduces the narrow (3, 17) event-count analysis, not every campaign-level diagnostic.

This separation is intentional: the prospective record is preserved as it was executed, later statistical repairs are visible as repairs, and the paper’s conclusions are limited to what the released evidence can support.

7 Data, code, and release status

The publication-oriented repository is [Vulkin-prog/vdw-rabung-bounds](#). It was exported under an explicit allowlist from the development repository at the immutable source commit `02796ae9e08da5b051a94b2d9001763908375f6e`. At the time of this revision it remains a private preparation repository, not a citable release. Its machine-readable status ledger deliberately leaves the certificate, campaign-recovery, ordered-prime, external-replication, rights, and DOI gates open. An immutable archive of the frozen release tag, rather than a moving branch, will be cited after those gates pass.

The source registry `audit/claims.json` contains exactly the thirteen succinct Rabung certificates (p, r, k) . The separate `audit/baselines.json` records published, archived, and current strict lower bounds with source locators. Running `tools/close_baselines.py` produces the recurrence-closed bounds, their provenance DAG, and the $\text{\LaTeX}$ table used in Section 4. This prevents a manually maintained record table from being compared with an unclosed bibliography.

For every direct claim, the final layout is

```
results/claims/<claim-id>/{manifest.json,raw/*}.
```

The manifest records the claim, verdict, max-run and (B^*) profiles, Git commit, source and binary hashes, build commands, compiler versions, and the SHA-256 and byte length of every raw standard stream. The fail-closed parser in `tools/claim_audit.py` validates those manifests

and generates the single thirteen-row audit table. The placeholder currently committed under `results/claims/` is not evidence and cannot close this gate.

The campaign record consists of 515 checkpoint rows and associated raw or derived per-chunk artifacts. Part of that material was historically ignored and must first be recovered read-only from the campaign machine, including chunk 7 from checkpoint evidence although its journal write is absent. The release also requires byte-level equality of the scanner and independent CPU ordered-prime streams for every chunk; matching congruence-class counts alone does not establish identity. The four exact historical scanner blobs are already preserved in the publication repository with their revision mapping.

The separate registry `audit/rescan17.json` binds the preserved post-campaign transcript for the first 17 chunks to its full Git commit, source blob, raw SHA-256, and eight independent counts committed before the run completed. A focused public provenance record contains those preregistered counts without importing the unrelated internal journal from which they were extracted. Running `tools/rescan17_audit.py` parses the historical format fail-closed and generates Table 4. Because the old run did not preserve separate standard streams, a binary hash, or per-chunk rows, the registry records those fields as unavailable rather than reconstructing them. This aggregate audit does not substitute for the 515-chunk ordered-prime identity gate.

The cyclic W_c pilot and its proof artifacts are outside the scientific scope of this records paper and are intentionally absent from this repository. The archived two-colour phase-2 source is credited and located, but a verbatim third-party web capture is not redistributed unless its rights status is resolved. These provenance metadata support priority attribution; they do not turn an archived claim of exhaustive coverage into a newly audited fact.

Before submission, a clean release candidate must rebuild the scanner and verifiers, execute and validate the thirteen claim manifests, regenerate every table, and compile this PDF. After the pre-freeze gates pass and the DOI metadata are final, the entire intended payload is staged. Immediately before the inner-freeze transaction, the staged release form of `STATUS.json` sets `repository_state` to `archived_release`, sets `citable_release` true, and marks every gate passed. For the DOI gate, that transient staged state means only that the metadata are final and the inner-freeze transaction is ready; it does not assert that a tag, archive, deposit, or receipt is already public.

The transaction `tools/freeze_release.py create` validates that staged candidate, rebuilds the PDF, and writes the top-level `MANIFEST.sha256`. This is the first checksum layer and binds the tracked payload files: the PDF, source files, certificates, logs, and data. If the transaction fails, the staged release state is neither committed nor published. Only after success is the manifest staged with the payload, which is then committed, tagged locally, and rechecked. A deterministic archive is subsequently created from that checked immutable local tag and rechecked byte-for-byte by `tools/build_release_archive.py`; this is the second checksum layer. The archive's own byte length and SHA-256, together with the post-tag draft-deposit comparison, belong in an external deposit receipt or release record. They cannot be inserted into the archive they hash without a cycle. The fail-closed utility `tools/deposit_receipt.py` creates and rechecks that JSON only at an absolute path outside the repository, and the receipt is not embedded evidence for the DOI gate.

Four fixed headline certificates must additionally be checked on independently administered hardware with complete outputs preserved. After the scientific, replication, and rights gates pass, the DOI can be reserved and inserted into the exact final candidate. The successful inner-freeze transaction precedes the commit and local tag; the checked tagged tree is then archived and

compared with the draft upload. The tag and deposit are made public only if that comparison succeeds. Until then this manuscript does not claim that a complete citable artifact is available.

A Equivalence of the operational criterion

The scan tests the criterion $(a) \wedge (B^*)$ of Section 2 rather than Rabung's $(a) \wedge (b)$ verbatim. The two are equivalent, and since every bound in this paper rests on that equivalence we prove it here rather than rely on the exhaustive machine comparison of Section 2, which we regard as a check on the *implementation*, not as a substitute for a proof.

Setting. Let $p \equiv 1 \pmod{r}$ with $p > k \geq 3$, let g be a primitive root, and colour $x \in [1, p-1]$ by $c(x) = \text{ind}_g(x) \pmod{r}$; this is the power-residue colouring, and $c(xy) = c(x) + c(y)$. Extend it to $[0, (k-1)p]$ by reducing modulo p and colouring the multiples of p (the *jokers*) by an arbitrary map $\vartheta' : \{0, p, \dots, (k-1)p\} \rightarrow \mathbb{Z}_r$. Write $\vartheta(-1) = c(p-1)$, which is 0 if -1 is an r -th power residue and $r/2$ otherwise (the latter requires r even).

An arithmetic progression of length k and common difference d is monochromatic iff its terms all receive one colour. Progressions avoiding the jokers are handled by (a): by complete multiplicativity of c , the progression $a, a+d, \dots, a+(k-1)d$ reduces, after dividing by d (which permutes the classes by the constant $c(d)$), to the window $\{a/d, a/d+1, \dots, a/d+k-1\}$ of k consecutive residues. Hence *some* joker-free progression is monochromatic iff *some* window of k consecutive residues in $[1, p-1]$ is, which is exactly the negation of (a).

The joker condition. It remains to treat the progressions that meet a joker. First bound the common difference: a k -term progression inside $[0, (k-1)p]$ has $(k-1)d \leq (k-1)p$, so $1 \leq d \leq p$, and the case $d = p$ occurs for exactly one progression, namely $0, p, 2p, \dots, (k-1)p$, which consists *entirely* of jokers. That progression is monochromatic if and only if ϑ' is constant—it is precisely the reason Rabung's statement requires a *non-constant* joker colouring, and any non-constant ϑ' breaks it. For the remaining case $0 < d < p$: since the progression has $k < p$ terms, two of them congruent to 0 modulo p would force $p \mid (i' - i)d$ with $0 < |i' - i| < k < p$ and $p \nmid d$, which is impossible; so the progression meets at most one multiple of p . Say it is the $(j+1)$ -st term, $0 \leq j \leq k-1$. Dividing by d as before, the progression becomes the *shifted string*

$$S_j = \{-j, \dots, -1\} \cup \{1, \dots, k-1-j\} \pmod{p},$$

centred on the joker, together with the joker itself.

Proposition 4. Fix $p \equiv 1 \pmod{r}$ and k . The colouring above is free of monochromatic k -term progressions for some choice of ϑ' if and only if $(a) \wedge (B^*)$ holds, where

$$(B^*) : \quad \text{no shifted string } S_j \text{ } (0 \leq j \leq k-1) \text{ is monochromatic.}$$

Moreover, when $(a) \wedge (B^*)$ holds, every non-constant ϑ' works; and when it fails, no ϑ' works. In particular the choice of ϑ' is immaterial, as asserted by Rabung.

Proof. Suppose some S_j is monochromatic, of colour w say.

The lift. We first check that every residue class of d is realised by a genuine progression inside $[0, (k-1)p]$ through a joker. Fix a representative $d \in [1, p-1]$ and take the joker to be jp and the first term to be

$$a = j(p-d).$$

Then $a + jd = jp$, so the $(j + 1)$ -st term is the joker; the first term $a = j(p - d) \geq 0$ since $d < p$; and the last term is

$$a + (k - 1)d = jp + (k - 1 - j)d \leq jp + (k - 1 - j)(p - 1) \leq (k - 1)p,$$

with equality in the last step only at $j = k - 1$, where that last term is the joker jp itself; so all k terms lie in $[0, (k - 1)p]$. The non-joker terms are $jp + id$ for $i \in S_j$, whose residues are $id \bmod p$; hence their colours are $c(i) + c(d) = w + c(d)$.

The all-or-nothing mechanism. As d ranges over $(\mathbb{Z}/p)^*$, the value $w + c(d)$ ranges over *all* of $\mathbb{Z}_r$ (because c is onto), and by the lift each such value is attained by an admissible progression through the *same, fixed* joker jp . Hence for each colour $u \in \mathbb{Z}_r$ there is a progression through jp whose other $k - 1$ terms are all of colour u ; assigning $\vartheta'(jp) = u$ makes it monochromatic. Every value of ϑ' at the single joker jp is thus forbidden, so no choice of ϑ' —constant or not—avoids a monochromatic progression. Conversely, if no S_j is monochromatic then no progression with $d < p$ through a joker has all its other $k - 1$ terms of a single colour, so no assignment of ϑ' can complete one; the only remaining progression is the all-joker one ($d = p$), which any *non-constant* ϑ' breaks; combined with (a) for the joker-free progressions, every non-constant ϑ' works. (An earlier version of this proof overlooked the all-joker progression and concluded that *any* ϑ' works; that is false for constant ϑ' , and the non-constancy in the Proposition is exactly what the $d = p$ case consumes. The condition $(a) \wedge (B^*)$ is thus all-or-nothing over the non-constant ϑ' , which is why Rabung may state it without reference to the joker colouring.) $\square$

Reduction of (B^*) to a single interval. Since $c(-x) = c(x) + \vartheta(-1)$, the string S_j is monochromatic iff the two intervals $\{1, \dots, j\}$ and $\{1, \dots, k - 1 - j\}$ are each monochromatic and their colours agree after the shift by $\vartheta(-1)$.

Case $\vartheta(-1) = 0$. Then $c(-x) = c(x)$, so S_j is monochromatic iff the interval $\{1, \dots, \max(j, k - 1 - j)\}$ is. Minimising $\max(j, k - 1 - j)$ over $0 \leq j \leq k - 1$ gives $\lceil (k - 1)/2 \rceil$, attained at $j = \lfloor (k - 1)/2 \rfloor$. Hence

$$(B^*) \iff \{1, 2, \dots, \lceil (k - 1)/2 \rceil\} \text{ is not monochromatic.}$$

A one-element interval *is* monochromatic, and therefore fails the condition; under the standing assumption $k \geq 3$ this is the case $k = 3$, and it is the reason a singleton must be counted as “all in the same class”.

Case $\vartheta(-1) = r/2$ (r even). Now $c(-x) = c(x) + r/2 \neq c(x)$, so a two-sided string can never be monochromatic: only the one-sided strings $j = 0$ and $j = k - 1$ survive, both giving the interval $\{1, \dots, k - 1\}$. Hence

$$(B^*) \iff \{1, 2, \dots, k - 1\} \text{ is not monochromatic,}$$

Unlike the length- k prohibition of (a), this is a condition on an interval of length $k - 1$, and is therefore *not* implied by (a): it is a genuine additional constraint, and on a longer interval than in the previous case.

Remark 5. Both refinements matter. Replacing $\lceil \cdot \rceil$ by $\lfloor \cdot \rfloor$, or treating a singleton as vacuously satisfying the condition, gives a boundary condition that is not equivalent to (b). Counterexamples occur already at $p = 13$, $r = 2$, $k = 4$ for the floor variant and at $p = 5$, $r = 2$, $k = 3$ for the singleton convention; condition (a) can mask some such boundary errors. An early version of our own tester made exactly these two errors, and the comparison against the oracle is what caught them.

Acknowledgements

This work rests on Daniel Monroe’s distributed computation [7] and on the code and direct tally released with it [5], which served as the baseline against which our engine was validated. We are further indebted to the unpublished second phase of that project, whose archived results we recovered, rechecked in historical runs, and report here with credit [6] (Section 4). We thank the maintainers of the Internet Archive, without which those results would have been lost.

AI contribution statement

The research reported here was carried out in collaboration with AI systems: Anthropic’s Claude (models `claude-fable-5` and `claude-opus-4-8`) and OpenAI Codex in ChatGPT Work. These systems contributed at a level well beyond copy-editing. Following the practice now emerging in this literature, we state precisely what the models did, what the author did, and—most importantly—how the results can be checked without trusting either.

What the AI systems did. Under the author’s direction, the Claude models wrote the GPU scanning engine (`scan_gpu.cu`) and the independent verifiers, designed and executed the campaign of Section 3, carried out the statistical analysis of Section 5, and produced the first draft of this paper. They also performed the pre-announcement literature scan discussed below. During the publication-finalization phase, OpenAI Codex developed and hardened parts of the reproducibility package and its release contracts, wrote tests and operator/recovery tooling, ran CPU-only checks, audited provenance and internal consistency, and proposed or drafted the corresponding manuscript revisions. It did not operate the historical campaign PC, execute the missing GPU release qualification, or supply external-replication evidence.

What the author did. The author conceived the project and set its scope; made every decision about what to compute, when to stop, and what to claim; provided and operated the hardware; and supplied adversarial challenges at each stage, several of which caught substantive errors that the automated review had missed. The author takes full responsibility for the entire contents of this paper, including the data, the references, and the conclusions, irrespective of how any part of it was generated.

How the results can be checked. We regard the following apparatus, and not any assurance about the tools used, as the basis on which the bounds can be relied upon once the release gates in Section 7 are complete.

- *Certificates outside the pipeline.* Each bound is certified by `verify_claim`, which recomputes the coloring from scratch in time $O(p \log p)$ and shares no code with the search engine. A reader who trusts nothing about our software can inspect or reimplement this verifier (Section 2 reports its measured cost on our hardware: seconds to minutes per bound, $O(p)$ memory).
- *Dependency-aware redundancy.* The mirror and fused full scans share source and arithmetic, so we treat them as redundant paths, not independent implementations. The principal

separation comes from the CPU criterion, `verify_claim`, and the Montgomery-free high-range witness; the exact dependency matrix is Table 2. Final verdicts are parsed fail-closed and their digests engage the claim identity rather than only a max-run profile.

- *Preregistration.* The quantitative forecasts of Section 5 were committed to the repository before the corresponding data existed, and read at four fixed cumulative checkpoints (Section 6). These readings form one correlated trajectory, not four replications. Full commit identifiers and derived public records are preserved in the audit registry; the underlying commits belong to the source repository history and are not asserted to be objects in this export’s Git database.
- *Novelty, not just correctness.* Because literature-search errors can invalidate novelty claims, we audited priority separately rather than assuming it. Our pre-announcement scan located an unpublished two-colour computation that had gone further than ours for $r = 2$; rather than claim those primes, we credit them to their author and report that our historical certificate runs rechecked them; final release manifests remain required (Section 4). Every bibliographic reference in the release bibliography is restricted to an entry actually cited by this paper. Each locator is checked against a publisher record or the primary document, and a machine-readable public qualification report records the source used for that check without embedding conversational review notes in the bibliography.

Raw traces. The campaign logs, structured claim manifests, source identities, and public qualification reports are required components of the immutable release. Internal conversational journals and adversarial-review transcripts are not scientific evidence and are excluded from the public artifact. As Section 7 records, the archive and its DOI have not yet been frozen; the private preparation repository is not represented as the final audit package.

References

- [1] Thomas Blankenship, Jay Cummings, and Vladislav Taranchuk. A new lower bound for van der Waerden numbers. *European Journal of Combinatorics*, 69:163–168, 2018. doi: 10.1016/j.ejc.2017.10.007. URL <https://doi.org/10.1016/j.ejc.2017.10.007>.
- [2] Jacob Fox and Zach Hunter. Three-color van der Waerden numbers grow super-exponentially. arXiv:2606.02541v1, June 2026. URL <https://arxiv.org/abs/2606.02541v1>. Version 1, 1 June 2026.
- [3] P. R. Herwig, M. J. H. Heule, P. M. van Lambalgen, and H. van Maaren. A new method to construct lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 14(1):R6, 2007. doi: 10.37236/925. URL <https://doi.org/10.37236/925>.
- [4] Meilian Liang, Xiaodong Xu, Zehui Shao, and Baoxin Xiu. Lower bounds on some van der Waerden numbers based on quadratic residues. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 83:23–31, 2012. URL <https://combinatorialpress.com/jcmcc-articles/volume-083/lower-bounds-on-some-van-der-waerden-numbers-based-on-quadratic-residues/>.
- [5] Daniel Monroe. Direct Rabung tally distributed with the van der Waerden search code. Versioned project data file, 2021. URL <https://github.com/>

- hmonroe/vdw/blob/f1223097f4adc0a41ccb92ace51ebafd2b996fb1/output.txt. Im-mutable Git commit f1223097f4adc0a41ccb92ace51ebafd2b996fb1; file output.txt; SHA-256 bd88e781acdceead847e0b736ca788cbc510aac744d025c2efbde4b152725774.
- [6] Daniel Monroe. BOINC phase-2 bounds table. Internet Archive capture of <http://vdwnumbers.org/bounds.txt>, 2021. URL https://web.archive.org/web/20220428101631id_/http://vdwnumbers.org/bounds.txt. Source header dated 18 August 2021; archived 28 April 2022; SHA-256 0e8f79232e3855cbc64026a4c2f3a82433c0508456e0c23f76eb1a400efeb556.
 - [7] Daniel Monroe. New lower bounds for Van der Waerden numbers using distributed computing. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 128:305–315, 2026. doi: 10.61091/jcmcc128-19. URL <https://doi.org/10.61091/jcmcc128-19>. Published online 4 December 2025.
 - [8] Daniel Monroe. Projects. Personal website, <https://daniel-monroe.github.io/projects.html>, 2026. URL <https://github.com/daniel-monroe/daniel-monroe.github.io/blob/825778cca5e669cbf60140e35170113603010076/projects.html>. Archived author-owned version dated 28 March 2026; live page accessed 28 August 2026.
 - [9] Tomás Oliveira e Silva. Tables of values of $\pi(x)$ and of $\pi_2(x)$. University of Aveiro; data file <https://sweet.ua.pt/tos/primes/1d09.txt.gz>, 2005. URL https://web.archive.org/web/20160704031247id_/http://sweet.ua.pt/tos/primes/1d09.txt.gz. Table of $\pi(k \cdot 10^9)$ last updated 23 January 2005; accessed 28 August 2026.
 - [10] John Rabung and Mark Lotts. Improving the use of cyclic zippers in finding lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 19(2):P35, 2012. doi: 10.37236/2363. URL <https://doi.org/10.37236/2363>.
 - [11] John R. Rabung. Some progression-free partitions constructed using Folkman’s method. *Canadian Mathematical Bulletin*, 22(1):87–91, 1979. doi: 10.4153/CMB-1979-012-1. URL <https://doi.org/10.4153/CMB-1979-012-1>.
 - [12] Jonathan Sorenson and Jonathan Webster. Strong pseudoprimes to twelve prime bases. *Mathematics of Computation*, 86(304):985–1003, 2017. doi: 10.1090/mcom/3134. URL <https://doi.org/10.1090/mcom/3134>.
 - [13] Xiaodong Xu. A generalization of sets without long arithmetic progressions based on Szekeres algorithm. *Journal of Number Theory*, 133(11):3670–3677, 2013. doi: 10.1016/j.jnt.2013.05.008. URL <https://doi.org/10.1016/j.jnt.2013.05.008>.